

// TUTORIAL //

Verwenden von Rsync zum Synchronisieren von lokalen und entfernten Verzeichnissen

Published on January 5, 2021

Backups

Linux Basics

Deutsch



Einführung

Rsync, was für „remote sync“ (entfernte Synchronisierung) steht, ist ein Tool zur Synchronisierung von lokalen und entfernten Dateien. Es verwendet einen Algorithmus, der die Menge der kopierten Daten minimiert, indem es nur die Abschnitte von Daten verschiebt, die Änderungen aufweisen.

In diesem Leitfaden werden wir die grundlegende Verwendung dieses leistungsfähigen Dienstprogramms abdecken.

Was ist Rsync?

Rsync ist ein sehr flexibles, netzwerkfähiges Synchronisierungstool. Aufgrund seiner Allgegenwärtigkeit unter Linux- und Unix-ähnlichen Systemen und seiner Beliebtheit als Tool für Systemskripte, ist es in den meisten Linux-Distributionen standardmäßig enthalten.

Grundlegende Syntax

Die grundlegende Syntax von `rsync` ist sehr einfach und funktioniert ähnlich wie `ssh`, `scp` und `cp`.

Wir erstellen zwei Testverzeichnisse und einige Testdateien mit den folgenden Befehlen:

```
$ cd ~  
$ mkdir dir1  
$ mkdir dir2  
$ touch dir1/file{1..100}
```

Wir haben nun ein Verzeichnis namens `dir1` mit 100 leeren Dateien.

```
$ ls dir1
```

Output

```
file1    file18  file27  file36  file45  file54  file63  file72  file81  fi  
file10   file19  file28  file37  file46  file55  file64  file73  file82  fi  
file100  file2   file29  file38  file47  file56  file65  file74  file83  fi  
file11   file20  file3   file39  file48  file57  file66  file75  file84  fi  
file12   file21  file30  file4   file49  file58  file67  file76  file85  fi  
file13   file22  file31  file40  file5   file59  file68  file77  file86  fi  
file14   file23  file32  file41  file50  file6   file69  file78  file87  fi  
file15   file24  file33  file42  file51  file60  file7   file79  file88  fi  
file16   file25  file34  file43  file52  file61  file70  file8   file89  fi  
file17   file26  file35  file44  file53  file62  file71  file80  file9   fi
```

Wir haben auch ein leeres Verzeichnis namens `dir2`.

Um den Inhalt von `dir1` mit `dir2` auf demselben System zu synchronisieren, geben

Sie Folgendes ein:

```
$ rsync -r dir1/ dir2
```

Die Option `-r` bedeutet rekursiv, das für die Verzeichnissynchronisation erforderlich ist.

Wir könnten stattdessen auch das Flag `-a` verwenden:

```
$ rsync -a dir1/ dir2
```

Die Option `-a` ist ein Kombinations-Flag. Sie steht für „Archiv“ und synchronisiert rekursiv und erhält symbolische Links, spezielle und Gerätedateien, Modifizierungszeiten, Gruppe, Eigentümer und Berechtigungen. Sie wird häufiger verwendet als `-r` und ist normalerweise das, was Sie verwenden möchten.

Ein wichtiger Hinweis

Vielleicht haben Sie bemerkt, dass es am Ende des ersten Arguments in den obigen Befehlen einen nachstehenden Schrägstrich (/) gibt:

```
$ rsync -a dir1 / dir2
```

Dies ist notwendig, um „den Inhalt von `dir1`“ zu bezeichnen. Die Alternative ohne den nachstehenden Schrägstrich würde `dir1`, einschließlich des Verzeichnisses, innerhalb von `dir2` platzieren. Dies würde eine Hierarchie erstellen, die wie folgt aussieht:

```
$ ~/dir2/dir1/[files]
```

Überprüfen Sie immer Ihre Argumente, bevor Sie einen Befehl `rsync` ausführen. `Rsync`

bietet eine Methode dafür, indem Sie die Optionen `-n` oder `--dry-run` übergeben. Das Flag `-v` (für „verbose“, ausführlich) ist ebenfalls notwendig, um die entsprechende Ausgabe zu erhalten:

```
$ rsync -anv dir1/ dir2
```

Output

```
sending incremental file list
./
file1
file10
file100
file11
file12
file13
file14
file15
file16
file17
file18
. . .
```

Vergleichen Sie diese Ausgabe mit der Ausgabe, die wir erhalten, wenn wir den nachgestellten Schrägstrich entfernen:

```
$ rsync -anv dir1 dir2
```

Output

```
sending incremental file list
dir1/
dir1/file1
dir1/file10
dir1/file100
dir1/file11
dir1/file12
dir1/file13
dir1/file14
dir1/file15
```

```
dir1/file16
dir1/file17
dir1/file18
. . .
```

Hier sehen Sie, dass das Verzeichnis selbst übertragen wird.

Verwenden von Rsync zum Synchronisieren mit einem Remote-System

Die Synchronisierung mit einem Remote-System ist trivial, wenn Sie SSH-Zugriff auf den Remote-Rechner haben und `rsync` auf beiden Seiten installiert ist. Sobald Sie SSH-Zugriff zwischen den beiden Rechnern verifiziert haben, können Sie den Ordner `dir1` von früher auf einen Remote-Computer synchronisieren, indem Sie diese Syntax verwenden (beachten Sie, dass wir in diesem Fall das eigentliche Verzeichnis übertragen *möchten*, daher lassen wir den nachgestellten Schrägstrich weg):

```
$ rsync -a ~/dir1 username @ remote_host : destination_directory
```

Dies wird als „push“-Operation bezeichnet, da es ein Verzeichnis vom lokalen System auf ein Remote-System schiebt. Die entgegengesetzte Operation ist „pull“. Sie wird verwendet, um ein Remote-Verzeichnis mit dem lokalen System zu synchronisieren. Wenn das `dir1` auf dem Remote-System anstelle unseres lokalen Systems wäre, würde die Syntax lauten:

```
$ rsync -a username @ remote_host : /home/username/dir1 place_to_sync_on_
```

Wie `cp` und ähnliche Tools ist die Quelle immer das erste Argument, und das Ziel immer das zweite.

Nützliche Optionen für Rsync

Rsync bietet viele Optionen zur Änderung des Standardverhaltens des Dienstprogramms. Wir haben bereits einige der notwendigen Flags besprochen.

Wenn Sie Dateien übertragen, die nicht bereits komprimiert wurden, wie Textdateien, können Sie die Netzwerkübertragung reduzieren, indem Sie mit der Option `-z` eine Komprimierung hinzufügen:

```
$ rsync -az source destination
```

Das Flag `-P` ist sehr hilfreich. Es kombiniert die Flags `--progress` und `--partial`. Mit dem ersten erhalten Sie eine Fortschrittsleiste für die Übertragungen und mit dem zweiten können Sie unterbrochene Übertragungen wiederaufnehmen:

```
$ rsync -azP source destination
```

Output

```
sending incremental file list
```

```
./
```

```
file1
```

```
0 100% 0.00kB/s 0:00:00 (xfer#1, to-check=99/101)
```

```
file10
```

```
0 100% 0.00kB/s 0:00:00 (xfer#2, to-check=98/101)
```

```
file100
```

```
0 100% 0.00kB/s 0:00:00 (xfer#3, to-check=97/101)
```

```
file11
```

```
0 100% 0.00kB/s 0:00:00 (xfer#4, to-check=96/101)
```

```
...
```

Wenn wir den Befehl erneut ausführen, erhalten wir eine kürzere Ausgabe, da keine Änderungen vorgenommen wurden. Dies illustriert die Fähigkeit von `rsync` anhand von Änderungszeiten, um zu ermitteln, ob Änderungen vorgenommen wurden.

```
$ rsync -azP source destination
```

Output

```
sending incremental file list
sent 818 bytes received 12 bytes 1660.00 bytes/sec
total size is 0 speedup is 0.00
```

Wir können die Änderungszeit einiger Dateien aktualisieren und sehen, dass rsync auf intelligente Weise nur die geänderten Dateien neu kopiert:

```
$ touch dir1/file{1..10}
$ rsync -azP source destination
```

Output

```
sending incremental file list
file1
      0 100%    0.00kB/s    0:00:00 (xfer#1, to-check=99/101)
file10
      0 100%    0.00kB/s    0:00:00 (xfer#2, to-check=98/101)
file2
      0 100%    0.00kB/s    0:00:00 (xfer#3, to-check=87/101)
file3
      0 100%    0.00kB/s    0:00:00 (xfer#4, to-check=76/101)
. . .
```

Um zwei Verzeichnisse wirklich synchron zu halten, ist es notwendig, Dateien aus dem Zielverzeichnis zu löschen, wenn sie aus der Quelle entfernt werden. Standardmäßig löscht rsync nichts aus dem Zielverzeichnis.

Wir können dieses Verhalten mit der Option `--delete` ändern. Bevor Sie diese Option verwenden, sollten Sie die Option `--dry-run` verwenden und Tests durchführen, um Datenverlust zu vermeiden:

```
$ rsync -a --delete source destination
```

Wenn Sie bestimmte Dateien oder Verzeichnisse ausschließen möchten, die sich innerhalb eines synchronisierten Verzeichnisses befinden, können Sie dies tun, indem Sie sie in einer kommasetrennten Liste nach der Option `--exclude=` angeben:

```
$ rsync -a --exclude= pattern_to_exclude source destination
```

Wenn wir ein Muster zum Ausschließen angegeben haben, können wir diesen Ausschluss für Dateien, die einem anderen Muster entsprechen, mit der Option `--include=` überschreiben.

```
$ rsync -a --exclude= pattern_to_exclude --include= pattern_to_include source destination
```

Schließlich kann die Option `--backup` von `rsync` verwendet werden, um Backups von wichtigen Dateien zu speichern. Sie wird in Verbindung mit der Option `--backup-dir` verwendet, die das Verzeichnis angibt, in dem die Backup-Dateien gespeichert werden sollen.

```
$ rsync -a --delete --backup --backup-dir= /path/to/backups /path/to/source
```

Zusammenfassung

Rsync kann Dateiübertragungen über Netzwerkverbindungen vereinfachen und die lokale Verzeichnissynchronisierung robuster machen. Die Flexibilität von `rsync` macht es zu einer guten Option für viele verschiedene Operationen auf Dateiebene.

Wenn Sie `rsync` beherrschen, können Sie komplexe Backup-Operationen entwerfen und eine feinkörnige Kontrolle darüber erhalten, was und wie übertragen wird.